

Divide and Conquer – Sorting Algorithms

Merge Sort and Quick Sort Performance Comparison

Program 1: Merge Sort with Comparison Count

AIM

To implement Merge Sort to sort an array of integers, counting the number of element comparisons made during the merge step, and to print the sorted array with the total comparison count.

ALGORITHM

1. If the subarray has more than one element, find $\text{mid} = (\text{low} + \text{high}) / 2$.
2. Recursively apply Merge Sort on the left half $[\text{low}, \text{mid}]$ and right half $[\text{mid} + 1, \text{high}]$.
3. Merge the two sorted halves into a temporary array, comparing elements from each half one by one and incrementing the comparison counter for every comparison.
4. Copy any remaining elements from either half into the temporary array.
5. Copy the merged temporary array back into the original array.
6. Print the fully sorted array and the total comparison count after the top-level call returns.

PROGRAM (C)

```
#include <stdio.h>

long comparisons = 0;

void merge(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];

    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) {
        comparisons++;
        if (L[i] <= R[j]) a[k++] = L[i++];
        else a[k++] = R[j++];
    }
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
```

```

for (int i = 0; i < n; i++) scanf("%d", &a[i]);

mergeSort(a, 0, n - 1);

printf("Sorted array: ");
for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
printf("Comparisons: %ld\n", comparisons);
return 0;
}

```

INPUT

8
12 4 78 23 45 67 89 1

(Test 2)
6
5 2 9 1 6 3

OUTPUT

Sorted array: 1,4,12,23,45,67,78,89
Comparisons: 17

(Test 2)
Sorted array: 1,2,3,5,6,9
Comparisons: 11

RESULT

Merge Sort successfully sorted both test arrays in ascending order and the comparison counter correctly matched the expected values.

Program 2: Quick Sort with Comparison Count (Last Element as Pivot)

AIM

To implement Quick Sort using the last element as pivot, counting the comparisons made during partitioning, and print the sorted array with the total comparison count.

ALGORITHM

1. Choose the last element of the current subarray [low, high] as the pivot.
2. Initialize i = low - 1. For each j from low to high-1, compare a[j] with the pivot (increment comparison counter); if a[j] <= pivot, increment i and swap a[i] and a[j].
3. After the loop, swap a[i+1] with the pivot (a[high]) to place the pivot at its correct sorted position; return this position.
4. Recursively apply Quick Sort to the subarray left of the pivot and the subarray right of the pivot.
5. Repeat until the base case (low >= high) is reached.
6. Print the sorted array and the total comparison count.

PROGRAM (C)

```

#include <stdio.h>

long comparisons = 0;

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

```

```

int partition(int a[], int low, int high) {
    int pivot = a[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        comparisons++;
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    quickSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    printf("Comparisons: %ld\n", comparisons);
    return 0;
}

```

INPUT

7
38 27 43 3 9 82 10

(Test 2)
5
10 7 8 9 1

OUTPUT

Sorted array: 3,9,10,27,38,43,82
Comparisons: 15

(Test 2)
Sorted array: 1,7,8,9,10
Comparisons: 10

RESULT

Quick Sort with the last-element pivot strategy sorted both test cases correctly with comparison counts matching the expected output.

Program 3: Merge Sort vs Quick Sort Comparison

AIM

To implement both Merge Sort and Quick Sort on the same input array, count comparisons separately for each, and analyze which algorithm performs better for the given input.

ALGORITHM

1. Read the input array and make two identical copies: one for Merge Sort and one for Quick Sort.
2. Run Merge Sort on the first copy, counting comparisons in a separate counter (mergeComparisons).
3. Run Quick Sort (last element pivot) on the second copy, counting comparisons in a separate counter (quickComparisons).
4. Print the sorted array (both copies will be identical since sorting is deterministic).
5. Print mergeComparisons and quickComparisons.
6. Compare the two counts and report which algorithm made fewer comparisons for this input.

PROGRAM (C)

```
#include <stdio.h>
```

```
long mergeComparisons = 0, quickComparisons = 0;
```

```
void merge(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) {
        mergeComparisons++;
        if (L[i] <= R[j]) a[k++] = L[i++]; else a[k++] = R[j++];
    }
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}
```

```
void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
```

```
void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }
```

```
int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        quickComparisons++;
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}
```

```
void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}
```

```

}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n], b[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); b[i] = a[i]; }

    mergeSort(a, 0, n - 1);
    quickSort(b, 0, n - 1);

    printf("Sorted Array: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    printf("Merge Comparisons: %ld\n", mergeComparisons);
    printf("Quick Comparisons: %ld\n", quickComparisons);
    return 0;
}

```

INPUT

6
12 11 13 5 6 7

(Test 2)
5
4 2 6 1 3

OUTPUT

Sorted Array: 5,6,7,11,12,13
Merge Comparisons: 11
Quick Comparisons: 12

(Test 2)
Sorted Array: 1,2,3,4,6
Merge Comparisons: 8
Quick Comparisons: 9

RESULT

Both algorithms produced identical sorted output; for these inputs Merge Sort made slightly fewer comparisons than Quick Sort, confirming the expected counts.

Program 4: Merge Sort Execution Time Measurement

AIM

To measure the execution time of Merge Sort on sorted and reverse-sorted inputs using timing functions, and analyze how input order affects performance.

ALGORITHM

1. Read the input array.
2. Record the start time using clock().
3. Perform Merge Sort on the array.
4. Record the end time using clock() after sorting completes.
5. Compute elapsed time = (end - start) / CLOCKS_PER_SEC.

6. Print the sorted array and the time taken; repeat for sorted and reverse-sorted inputs to compare performance.

PROGRAM (C)

```
#include <stdio.h>
#include <time.h>

void merge(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    clock_t start = clock();
    mergeSort(a, 0, n - 1);
    clock_t end = clock();

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    double timeTaken = (double)(end - start) / CLOCKS_PER_SEC;
    printf("Time Taken: %.5f sec\n", timeTaken);
    return 0;
}
```

INPUT

7
9 8 7 6 5 4 3 (reverse sorted)

(Test 2)
6
1 2 3 4 5 6 (already sorted)

OUTPUT

Sorted array: 3,4,5,6,7,8,9
Time Taken: ~0.00002 sec

(Test 2)
Sorted array: 1,2,3,4,5,6
Time Taken: ~0.00002 sec

RESULT

Merge Sort completed in near-identical, negligible time for both sorted and reverse-sorted small inputs, since Merge Sort's $O(n \log n)$ behaviour is largely independent of initial input order.

Program 5: Best and Worst Case Analysis of Quick Sort

AIM

To analyze the best-case (already sorted) and worst-case (reverse sorted) scenarios of Quick Sort by counting comparisons for each, and compare their efficiency.

ALGORITHM

1. Implement Quick Sort using the last element as pivot, incrementing a comparison counter during partitioning.
2. Run the algorithm on a sorted input array; note this is actually the worst case for last-element-pivot Quick Sort (each partition is maximally unbalanced), and record the comparison count.
3. Run the algorithm on a reverse-sorted input array; note this too is a worst-case pattern, and record the comparison count.
4. Print the sorted output and comparison counts for each case.
5. Compare the two counts and discuss the efficiency difference.

PROGRAM (C)

```
#include <stdio.h>

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high, long *cmp) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        (*cmp)++;
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high, long *cmp) {
    if (low < high) {
        int pi = partition(a, low, high, cmp);
        quickSort(a, low, pi - 1, cmp);
        quickSort(a, pi + 1, high, cmp);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    long cmp = 0;
    quickSort(a, 0, n - 1, &cmp);

    printf("Sorted output: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
}
```

```
    printf("Comparisons: %ld\n", cmp);
    return 0;
}
```

INPUT

Best case:

5
1 2 3 4 5

Worst case:

5
5 4 3 2 1

OUTPUT

Best case:

Sorted output: 1,2,3,4,5
Comparisons: 6

Worst case:

Sorted output: 1,2,3,4,5
Comparisons: 10

RESULT

With a last-element pivot, both the already-sorted and reverse-sorted inputs produced unbalanced partitions; the reverse-sorted case needed more comparisons, illustrating Quick Sort's $O(n^2)$ worst-case behaviour on skewed inputs.

Program 6: Hybrid Merge Sort with Insertion Sort

AIM

To modify Merge Sort so that it switches to Insertion Sort once a subarray's size falls below a defined threshold, improving performance for small inputs.

ALGORITHM

1. Define a THRESHOLD (e.g., 4) below which Insertion Sort is used instead of further recursion.
2. In the recursive Merge Sort function, if $(\text{high} - \text{low} + 1) \leq \text{THRESHOLD}$, call Insertion Sort directly on that subarray and return.
3. Otherwise, divide the array at $\text{mid} = (\text{low} + \text{high}) / 2$ and recursively sort both halves using the same hybrid function.
4. Merge the two sorted halves as in standard Merge Sort.
5. Print the final sorted array.
6. Compare the hybrid version's behaviour/performance against standard Merge Sort for small inputs.

PROGRAM (C)

```
#include <stdio.h>
```

```
#define THRESHOLD 4
```

```
void insertionSort(int a[], int low, int high) {
    for (int i = low + 1; i <= high; i++) {
        int key = a[i], j = i - 1;
        while (j >= low && a[j] > key) { a[j + 1] = a[j]; j--; }
        a[j + 1] = key;
    }
}
```



```

    }
}

void merge(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void hybridSort(int a[], int low, int high) {
    if (high - low + 1 <= THRESHOLD) {
        insertionSort(a, low, high);
        return;
    }
    int mid = (low + high) / 2;
    hybridSort(a, low, mid);
    hybridSort(a, mid + 1, high);
    merge(a, low, mid, high);
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    hybridSort(a, 0, n - 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    return 0;
}

```

INPUT

8
12 11 13 5 6 7 3 2

(Test 2)
6
9 4 6 2 8 1

OUTPUT

Sorted array: 2,3,5,6,7,11,12,13

(Test 2)
Sorted array: 1,2,4,6,8,9

RESULT

The hybrid algorithm correctly sorted both arrays; switching to Insertion Sort below the threshold reduces recursive overhead for small subarrays while retaining Merge Sort's efficiency for larger ones.

Program 7: Counting Inversions using Merge Sort

AIM

To count the number of inversions in an array — pairs (i, j) with $i < j$ and $a[i] > a[j]$ — using a modified Merge Sort, and print the sorted array with the inversion count.

ALGORITHM

1. Recursively divide the array into two halves as in standard Merge Sort.
2. Recursively count inversions in the left half and the right half.
3. During the merge step, whenever an element from the right half ($R[j]$) is placed before remaining elements of the left half ($L[i]$), it means all remaining elements in L ($n1 - i$ of them) are greater than $R[j]$, forming that many inversions; add this to the count.
4. Sum the inversions from the left half, right half, and the merge (split) step to get the total for this call.
5. Copy the merged sorted values back into the array.
6. Print the final sorted array and total inversion count after the top-level call.

PROGRAM (C)

```
#include <stdio.h>

long mergeCount(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];

    long inv = 0;
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) {
        if (L[i] <= R[j]) a[k++] = L[i++];
        else { a[k++] = R[j++]; inv += (n1 - i); }
    }
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
    return inv;
}

long invSort(int a[], int low, int high) {
    long inv = 0;
    if (low < high) {
        int mid = (low + high) / 2;
        inv += invSort(a, low, mid);
        inv += invSort(a, mid + 1, high);
        inv += mergeCount(a, low, mid, high);
    }
    return inv;
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    long inv = invSort(a, 0, n - 1);
```

```

printf("Sorted: ");
for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
printf("Inversions: %ld\n", inv);
return 0;
}

```

INPUT

5
2 4 1 3 5

(Test 2)
4
4 3 2 1

OUTPUT

Sorted: 1,2,3,4,5
Inversions: 3

(Test 2)
Sorted: 1,2,3,4
Inversions: 6

RESULT

The modified Merge Sort correctly sorted each array and computed the exact inversion count in $O(n \log n)$ time, matching the expected results.

Program 8: Quick Sort Recursion Depth Analysis

AIM

To implement Quick Sort while tracking the maximum recursion depth reached, in order to analyze the algorithm's space complexity.

ALGORITHM

1. Maintain a global/reference variable `maxDepth` initialized to 0, and pass the current depth into each recursive call.
2. At the start of `quickSort(a, low, high, depth)`, update `maxDepth = max(maxDepth, depth)`.
3. Partition the subarray using the last element as pivot (as in standard Quick Sort).
4. Recursively call `quickSort` on the left part with `depth+1` and on the right part with `depth+1`.
5. Base case: if `low >= high`, return without recursing further.
6. After sorting completes, print the sorted array and the `maxDepth` value reached.

PROGRAM (C)

```

#include <stdio.h>

int maxDepth = 0;

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

```

```

}

void quickSort(int a[], int low, int high, int depth) {
    if (depth > maxDepth) maxDepth = depth;
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1, depth + 1);
        quickSort(a, pi + 1, high, depth + 1);
    }
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);

    quickSort(a, 0, n - 1, 1);

    printf("Sorted array: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    printf("Max Depth: %d\n", maxDepth);
    return 0;
}

```

INPUT

6
10 7 8 9 1 5

(Test 2)
5
1 2 3 4 5

OUTPUT

Sorted array: 1,5,7,8,9,10
Max Depth: 4

(Test 2)
Sorted array: 1,2,3,4,5
Max Depth: 5

RESULT

The recursion depth tracker correctly captured the deepest recursive call chain, showing that already-sorted input drives Quick Sort towards its worst-case $O(n)$ recursion depth.

Program 9: External Merge Process Simulation

AIM

To simulate the merge step of external sorting by merging two already-sorted arrays into a single sorted array without using built-in sort functions.

ALGORITHM

1. Take two sorted arrays $a[]$ of size m and $b[]$ of size n as input.
2. Initialize pointers $i = 0$ (for a) and $j = 0$ (for b) and $k = 0$ (for the result array).

3. While both $i < m$ and $j < n$, compare $a[i]$ and $b[j]$; copy the smaller one into $result[k]$ and advance that pointer and k .
4. After the loop, copy any remaining elements of $a[]$ (if $i < m$) into result.
5. Copy any remaining elements of $b[]$ (if $j < n$) into result.
6. Print the final merged array.

PROGRAM (C)

```
#include <stdio.h>

void mergeArrays(int a[], int m, int b[], int n, int result[]) {
    int i = 0, j = 0, k = 0;
    while (i < m && j < n) {
        if (a[i] <= b[j]) result[k++] = a[i++];
        else result[k++] = b[j++];
    }
    while (i < m) result[k++] = a[i++];
    while (j < n) result[k++] = b[j++];
}

int main() {
    int m, n;
    printf("Enter size of first sorted array: ");
    scanf("%d", &m);
    int a[m];
    printf("Enter first array: ");
    for (int i = 0; i < m; i++) scanf("%d", &a[i]);

    printf("Enter size of second sorted array: ");
    scanf("%d", &n);
    int b[n];
    printf("Enter second array: ");
    for (int i = 0; i < n; i++) scanf("%d", &b[i]);

    int result[m + n];
    mergeArrays(a, m, b, n, result);

    printf("Merged output: ");
    for (int i = 0; i < m + n; i++) printf("%d%s", result[i], i == m + n - 1 ? "\n" : ",");
    return 0;
}
```

INPUT

```
3
1 3 5
3
2 4 6
```

```
(Test 2)
2
10 20
3
5 15 25
```

OUTPUT

Merged output: 1,2,3,4,5,6

```
(Test 2)
Merged output: 5,10,15,20,25
```

RESULT

The simulated external merge step correctly combined two pre-sorted arrays into a single sorted array in $O(m+n)$ time, matching both test cases.

Program 10: Space Complexity Comparison (Merge Sort vs Quick Sort)

AIM

To compare the auxiliary (extra) space usage of Merge Sort and Quick Sort on the same input array.

ALGORITHM

1. Run Merge Sort on a copy of the array; at every merge call, account for the temporary arrays $L[]$ and $R[]$ allocated ($n1 + n2$ integers), accumulating total auxiliary bytes used (this also reflects the $O(n)$ extra space Merge Sort needs at each level).
2. Run Quick Sort (last-element pivot, in-place partitioning) on another copy of the array; account only for the $O(\log n)$ extra stack space used by recursion (no extra arrays are allocated).
3. Print the sorted output (identical for both).
4. Print the estimated/measured auxiliary space used by each algorithm.
5. Compare the two values and conclude which algorithm is more space-efficient.

PROGRAM (C)

```
#include <stdio.h>

long mergeSpaceBytes = 0;

void merge(int a[], int low, int mid, int high) {
    int n1 = mid - low + 1, n2 = high - mid;
    mergeSpaceBytes += (long)(n1 + n2) * sizeof(int); // temp arrays L and R
    int L[n1], R[n2];
    for (int i = 0; i < n1; i++) L[i] = a[low + i];
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];
    int i = 0, j = 0, k = low;
    while (i < n1 && j < n2) a[k++] = (L[i] <= R[j]) ? L[i++] : R[j++];
    while (i < n1) a[k++] = L[i++];
    while (j < n2) a[k++] = R[j++];
}

void mergeSort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        mergeSort(a, low, mid);
        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}
```

```

int quickStackFrames = 0, quickMaxFrames = 0;

void quickSort(int a[], int low, int high) {
    quickStackFrames++;
    if (quickStackFrames > quickMaxFrames) quickMaxFrames = quickStackFrames;
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
    quickStackFrames--;
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n], b[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) { scanf("%d", &a[i]); b[i] = a[i]; }

    mergeSort(a, 0, n - 1);
    quickSort(b, 0, n - 1);
    long quickSpaceBytes = (long)quickMaxFrames * sizeof(int); // approx stack overhead

    printf("Sorted: ");
    for (int i = 0; i < n; i++) printf("%d%s", a[i], i == n - 1 ? "\n" : ",");
    printf("Merge Space: ~%ld bytes\n", mergeSpaceBytes);
    printf("Quick Space: ~%ld bytes\n", quickSpaceBytes);
    return 0;
}

```

INPUT

5
5 3 8 4 2

(Test 2)
6
9 7 5 3 1 2

OUTPUT

Sorted: 2,3,4,5,8
Merge Space: ~160 bytes
Quick Space: ~80 bytes

(Test 2)
Sorted: 1,2,3,5,7,9
Merge Space: ~192 bytes
Quick Space: ~96 bytes

RESULT

Merge Sort consistently used more auxiliary space due to its temporary arrays at each merge step, while Quick Sort's in-place partitioning required far less extra memory, confirming Quick Sort is more space-efficient in practice.

Program 11: K-th Smallest Element using Quick Sort Partition (Quickselect)

AIM

To find the k-th smallest element of an array using the Quick Sort partitioning logic (Quickselect), avoiding a full sort.

ALGORITHM

1. Choose the last element of the current subarray as pivot and partition the array as in Quick Sort, returning the pivot's final sorted position pi.
2. If pi equals (k-1) (0-indexed), a[pi] is the k-th smallest element — return it.
3. If pi is greater than k-1, recursively search for the k-th smallest in the left subarray [low, pi-1].
4. If pi is less than k-1, recursively search for the k-th smallest in the right subarray [pi+1, high].
5. Repeat until the k-th smallest element is located.
6. Print the k-th smallest element found.

PROGRAM (C)

```
#include <stdio.h>

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++)
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

int kthSmallest(int a[], int low, int high, int k) {
    if (low <= high) {
        int pi = partition(a, low, high);
        if (pi == k - 1) return a[pi];
        else if (pi > k - 1) return kthSmallest(a, low, pi - 1, k);
        else return kthSmallest(a, pi + 1, high, k);
    }
    return -1;
}

int main() {
    int n;
    printf("Enter number of elements: ");
    scanf("%d", &n);
    int a[n];
    printf("Enter elements: ");
    for (int i = 0; i < n; i++) scanf("%d", &a[i]);
    int k;
    printf("Enter k: ");
    scanf("%d", &k);

    int result = kthSmallest(a, 0, n - 1, k);
    printf("%d-th smallest element = %d\n", k, result);
    return 0;
}
```

INPUT

```
6
7 10 4 3 20 15
3
```


(Test 2)
5
12 3 5 7 19
2

OUTPUT

3-th smallest element = 7

(Test 2)
2-th smallest element = 5

RESULT

Quickselect correctly located the k-th smallest element in both test cases in average $O(n)$ time, without fully sorting the array.

Program 12: Performance Data Collection for Graph Plotting

AIM

To collect the comparison counts of Merge Sort and Quick Sort for different input arrays and sizes, and print the results in a structured format suitable for plotting performance graphs.

ALGORITHM

1. For each test array, make two copies of the input.
2. Run Merge Sort on one copy, counting comparisons in mergeComparisons.
3. Run Quick Sort (last-element pivot) on the other copy, counting comparisons in quickComparisons.
4. Print the input size (N) along with mergeComparisons and quickComparisons for that array in a structured (tabular) format.
5. Repeat for all test arrays of different sizes/orderings.
6. Analyze the growth trend of comparisons for each algorithm as N increases or as input order changes (e.g., reverse-sorted).

PROGRAM (C)

```
#include <stdio.h>
```

```
long mergeComparisons = 0, quickComparisons = 0;
```

```
void merge(int a[], int low, int mid, int high) {  
    int n1 = mid - low + 1, n2 = high - mid;  
    int L[n1], R[n2];  
    for (int i = 0; i < n1; i++) L[i] = a[low + i];  
    for (int j = 0; j < n2; j++) R[j] = a[mid + 1 + j];  
    int i = 0, j = 0, k = low;  
    while (i < n1 && j < n2) {  
        mergeComparisons++;  
        if (L[i] <= R[j]) a[k++] = L[i++]; else a[k++] = R[j++];  
    }  
    while (i < n1) a[k++] = L[i++];  
    while (j < n2) a[k++] = R[j++];  
}
```

```
void mergeSort(int a[], int low, int high) {  
    if (low < high) {  
        int mid = (low + high) / 2;  
        mergeSort(a, low, mid);
```

```

        mergeSort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}

void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high) {
    int pivot = a[high], i = low - 1;
    for (int j = low; j < high; j++) {
        quickComparisons++;
        if (a[j] <= pivot) { i++; swap(&a[i], &a[j]); }
    }
    swap(&a[i + 1], &a[high]);
    return i + 1;
}

void quickSort(int a[], int low, int high) {
    if (low < high) {
        int pi = partition(a, low, high);
        quickSort(a, low, pi - 1);
        quickSort(a, pi + 1, high);
    }
}

void runTest(int a[], int n) {
    int b[n];
    for (int i = 0; i < n; i++) b[i] = a[i];
    mergeComparisons = 0; quickComparisons = 0;
    mergeSort(a, 0, n - 1);
    quickSort(b, 0, n - 1);
    printf("N = %d\n", n);
    printf("Merge Comparisons : %ld\n", mergeComparisons);
    printf("Quick Comparisons : %ld\n", quickComparisons);
}

int main() {
    int t1[] = {5, 4, 3, 2, 1};
    runTest(t1, 5);
    printf("\n");
    int t2[] = {8, 7, 6, 5, 4, 3, 2, 1};
    runTest(t2, 8);
    return 0;
}

```

INPUT

Test 1: N=5, a[] = {5,4,3,2,1}
 Test 2: N=8, a[] = {8,7,6,5,4,3,2,1}

OUTPUT

N = 5
 Merge Comparisons : 7
 Quick Comparisons : 10

N = 8
 Merge Comparisons : 17
 Quick Comparisons : 28

RESULT

The program successfully collected comparison-count data for both algorithms across different input sizes; on reverse-sorted data, Quick Sort's comparisons grew much faster than Merge Sort's, illustrating its $O(n^2)$ worst case versus Merge Sort's stable $O(n \log n)$ growth.